Express.js

Základy

DELTA - Střední škola informatiky a ekonomie, s.r.o.

Ing. Luboš Zápotočný

20.10.2026

CC BY-NC-SA 4.0

Úvod

Webový framework

Co si představíte pod pojmem **webový framework**?

Webový framework

Co si představíte pod pojmem **webový framework**?

Znáte nějaké webové frameworky pro jiné jazyky?

Express.js

Express.js je

Express.js

Express.js je minimalistický a flexibilní webový framework pro Node.js

Express.js

Express.js je minimalistický a flexibilní webový framework pro Node.js

- Umožňuje vytvářet **webové aplikace** a **API**

Express.js

Express.js je minimalistický a flexibilní webový framework pro Node.js

- Umožňuje vytvářet **webové aplikace** a **API**
- Je **nejpopulárnějším** Node.js frameworkem

Express.js

Express.js je minimalistický a flexibilní webový framework pro Node.js

- Umožňuje vytvářet **webové aplikace** a **API**
- Je **nejpopulárnějším** Node.js frameworkem
- Poskytuje **jednoduché rozhraní** pro práci s HTTP

Express.js

Express.js je minimalistický a flexibilní webový framework pro Node.js

- Umožňuje vytvářet **webové aplikace** a **API**
- Je **nejpopulárnějším** Node.js frameworkem
- Poskytuje **jednoduché rozhraní** pro práci s HTTP
- Je základem pro mnoho dalších frameworků (NestJS, …)

Express.js

Proč používat Express.js?

Express.js

Proč používat Express.js?

- **Minimalistický** - obsahuje pouze základní funkce

Express.js

Proč používat Express.js?

- **Minimalistický** - obsahuje pouze základní funkce
- **Flexibilní** - můžete si přidat co potřebujete

Express.js

Proč používat Express.js?

- **Minimalistický** - obsahuje pouze základní funkce
- **Flexibilní** - můžete si přidat co potřebujete
- **Rychlý** - nízká režie oproti větším frameworkům

Express.js

Proč používat Express.js?

- **Minimalistický** - obsahuje pouze základní funkce
- **Flexibilní** - můžete si přidat co potřebujete
- **Rychlý** - nízká režie oproti větším frameworkům
- **Obrovská komunita** - spousta middleware a tutoriálů

Express.js

Proč používat Express.js?

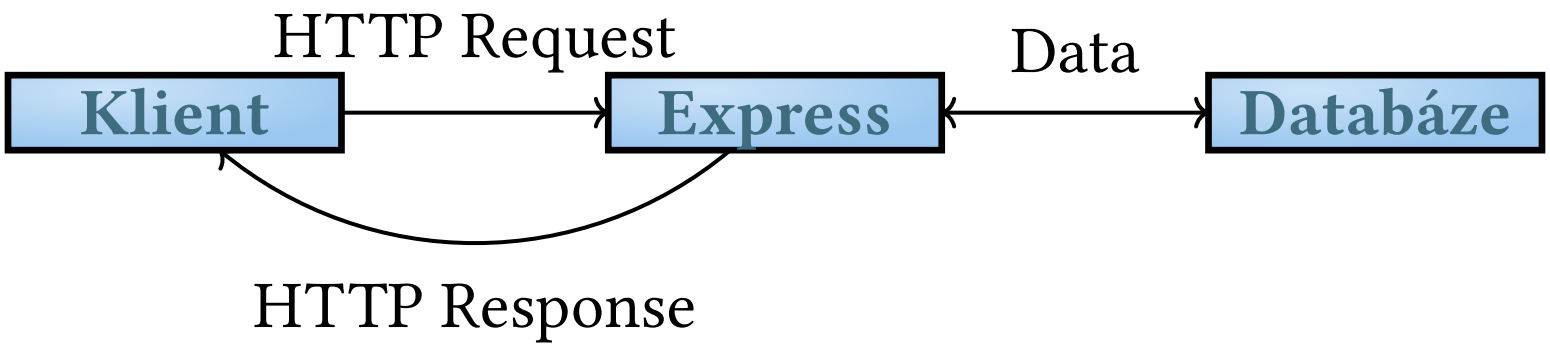
- **Minimalistický** - obsahuje pouze základní funkce
- **Flexibilní** - můžete si přidat co potřebujete
- **Rychlý** - nízká režie oproti větším frameworkům
- **Obrovská komunita** - spousta middleware a tutoriálů
- **Dobře zdokumentovaný** - kvalitní dokumentace

Express.js

Express.js architektura:

Express.js

Express.js architektura:



Instalace a první aplikace

Vytvoření nového Node.js projektu:

```
mkdir my-express-app
cd my-express-app
npm init -y
```

Instalace a první aplikace

Vytvoření nového Node.js projektu:

```
mkdir my-express-app
cd my-express-app
npm init -y
```

Instalace Express.js:

```
npm install express
```

Instalace a první aplikace

Hello World v Express.js:

```
const express = require('express');
const app = express();

app.get('/', (req, res) => {
  res.send('Hello World!');
});

app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

Instalace a první aplikace

Spuštění aplikace:

```
node index.js
```

Instalace a první aplikace

Spuštění aplikace:

```
node index.js
```

Výstup:

```
Server running on http://localhost:3000
```

Instalace a první aplikace

Spuštění aplikace:

```
node index.js
```

Výstup:

```
Server running on http://localhost:3000
```

Nyní můžete otevřít prohlížeč na adrese `http://localhost:3000`

Instalace a první aplikace

Co se děje v kódu?

Instalace a první aplikace

Co se děje v kódu?

1. `require('express')` - importujeme Express

Instalace a první aplikace

Co se děje v kódu?

1. `require('express')` - importujeme Express
2. `express()` - vytvoříme aplikaci

Instalace a první aplikace

Co se děje v kódu?

1. `require('express')` - importujeme Express
2. `express()` - vytvoříme aplikaci
3. `app.get('/', ...)` - definujeme route pro GET požadavky na `/`

Instalace a první aplikace

Co se děje v kódu?

1. `require('express')` - importujeme Express
2. `express()` - vytvoříme aplikaci
3. `app.get('/', ...)` - definujeme route pro GET požadavky na `/`
4. `res.send(...)` - odešleme odpověď klientovi

Instalace a první aplikace

Co se děje v kódu?

1. `require('express')` - importujeme Express
2. `express()` - vytvoříme aplikaci
3. `app.get('/', ...)` - definujeme route pro GET požadavky na `/`
4. `res.send(...)` - odešleme odpověď klientovi
5. `app.listen(3000, ...)` - spustíme server na portu 3000

Základy Express.js

Routing

Routing určuje,

Routing

Routing určuje, jak aplikace odpovídá na požadavky klienta na konkrétní endpoint

Routing

Routing určuje, jak aplikace odpovídá na požadavky klienta na konkrétní endpoint

Každá route může mít jednu nebo více handler funkcí

Routing

Základní HTTP metody:

```
// GET - retrieve data
app.get('/users', (req, res) => {
  res.send('Get all users');
});
```

Routing

Základní HTTP metody:

```
// GET - retrieve data
app.get('/users', (req, res) => {
  res.send('Get all users');
});

// POST - create data
app.post('/users', (req, res) => {
  console.log(req.body);
  res.send('Create user');
});
```

Routing

```
// PUT - update data
app.put('/users/:id', (req, res) => {
  console.log(req.body);
  res.send('Update user ' + req.params.id);
});
```

Routing

```
// PUT - update data
app.put('/users/:id', (req, res) => {
  console.log(req.body);
  res.send('Update user ' + req.params.id);
});

// DELETE - delete data
app.delete('/users/:id', (req, res) => {
  res.send('Delete user ' + req.params.id);
});
```

Routing

Route parametry umožňují zachytit hodnoty z URL:

```
app.get('/users/:id', (req, res) => {
  const userId = req.params.id;
  res.send('User ID: ' + userId);
});
```

Routing

Route parametry umožňují zachytit hodnoty z URL:

```
app.get('/users/:id', (req, res) => {
  const userId = req.params.id;
  res.send('User ID: ' + userId);
});
```

Požadavek na `/users/42` vrátí `User ID: 42`

Routing

Query parametry se předávají za ? v URL:

```
app.get('/search', (req, res) => {
  const query = req.query.q;
  const limit = req.query.limit || 10;
  res.send('Searching for: ' + query);
});
```

Routing

Query parametry se předávají za `?` v URL:

```
app.get('/search', (req, res) => {
  const query = req.query.q;
  const limit = req.query.limit || 10;
  res.send('Searching for: ' + query);
});
```

Požadavek na `/search?q=express&limit=5` vrátí

```
Searching for: express
```

Request a Response

Request objekt (`req`) obsahuje informace o příchozím požadavku:

Request a Response

Request objekt (`req`) obsahuje informace o příchozím požadavku:

- `req.params` - route parametry (`/users/:id`)

Request a Response

Request objekt (`req`) obsahuje informace o příchozím požadavku:

- `req.params` - route parametry (`/users/:id`)
- `req.query` - query string (`?name=John`)

Request a Response

Request objekt (`req`) obsahuje informace o příchozím požadavku:

- `req.params` - route parametry (`/users/:id`)
- `req.query` - query string (`?name=John`)
- `req.body` - tělo požadavku (pro POST/PUT)

Request a Response

Request objekt (`req`) obsahuje informace o příchozím požadavku:

- `req.params` - route parametry (`/users/:id`)
- `req.query` - query string (`?name=John`)
- `req.body` - tělo požadavku (pro POST/PUT)
- `req.headers` - HTTP hlavičky

Request a Response

Request objekt (`req`) obsahuje informace o příchozím požadavku:

- `req.params` - route parametry (`/users/:id`)
- `req.query` - query string (`?name=John`)
- `req.body` - tělo požadavku (pro POST/PUT)
- `req.headers` - HTTP hlavičky
- `req.method` - HTTP metoda (GET, POST, ...)

Request a Response

Request objekt (`req`) obsahuje informace o příchozím požadavku:

- `req.params` - route parametry (`/users/:id`)
- `req.query` - query string (`?name=John`)
- `req.body` - tělo požadavku (pro POST/PUT)
- `req.headers` - HTTP hlavičky
- `req.method` - HTTP metoda (GET, POST, ...)
- `req.path` - cesta URL

Request a Response

Response objekt (`res`) umožňuje odeslat odpověď:

Request a Response

Response objekt (`res`) umožňuje odeslat odpověď:

- `res.send()` - odešle text nebo HTML

Request a Response

Response objekt (`res`) umožňuje odeslat odpověď:

- `res.send()` - odešle text nebo HTML
- `res.json()` - odešle JSON data

Request a Response

Response objekt (`res`) umožňuje odeslat odpověď:

- `res.send()` - odešle text nebo HTML
- `res.json()` - odešle JSON data
- `res.status()` - nastaví HTTP status kód

Request a Response

Response objekt (`res`) umožňuje odeslat odpověď:

- `res.send()` - odešle text nebo HTML
- `res.json()` - odešle JSON data
- `res.status()` - nastaví HTTP status kód
- `res.redirect()` - přesměruje na jinou URL

Request a Response

Response objekt (`res`) umožňuje odeslat odpověď:

- `res.send()` - odešle text nebo HTML
- `res.json()` - odešle JSON data
- `res.status()` - nastaví HTTP status kód
- `res.redirect()` - přesměruje na jinou URL
- `res.sendFile()` - odešle soubor

Request a Response

Příklady odpovědí:

```
// Text response
res.send('Hello World');

// JSON response
res.json({ message: 'Hello', status: 'ok' });

// Status code with response
res.status(201).json({ id: 1, name: 'John' });
```

Request a Response

```
// Redirect
res.redirect('/login');

// Redirect with different status code
res.redirect(301, '/new-url');

// Send file
res.sendFile('/path/to/file.pdf');
```

Statické soubory

Express může servírovat **statické soubory** (HTML, CSS, JS, obrázky):

```
const express = require('express');
const app = express();

// Serve static files from 'public' folder
app.use(express.static('public'));

app.listen(3000);
```

Statické soubory

Express může servírovat **statické soubory** (HTML, CSS, JS, obrázky):

```
const express = require('express');
const app = express();

// Serve static files from 'public' folder
app.use(express.static('public'));

app.listen(3000);
```

Soubor `public/style.css` bude dostupný na `/style.css`

Statické soubory

Můžete přidat prefix pro statické soubory:

```
app.use('/static', express.static('public'));
```

Statické soubory

Můžete přidat prefix pro statické soubory:

```
app.use('/static', express.static('public'));
```

Soubor `public/style.css` bude dostupný na `/static/style.css`

Middleware

Co je middleware?

Middleware jsou

Co je middleware?

Middleware jsou funkce, které mají přístup k request a response objektům

Co je middleware?

Middleware jsou funkce, které mají přístup k request a response objektům

- Mohou **modifikovat** request a response

Co je middleware?

Middleware jsou funkce, které mají přístup k request a response objektům

- Mohou **modifikovat** request a response
- Mohou **ukončit** request-response cyklus

Co je middleware?

Middleware jsou funkce, které mají přístup k request a response objektům

- Mohou **modifikovat** request a response
- Mohou **ukončit** request-response cyklus
- Mohou **předat řízení** další middleware funkci

Co je middleware?



Co je middleware?



Middleware funkce se volají postupně pomocí `next()`

Co je middleware?

Základní struktura middleware:

```
function myMiddleware(req, res, next) {
  // Do something with req or res
  console.log('Middleware executed');

  // Pass control to next middleware
  next();
}
```

Vestavěné middleware

Express poskytuje několik vestavěných middleware:

Vestavěné middleware

Express poskytuje několik vestavěných middleware:

```
// Parse JSON bodies
app.use(express.json());

// Parse URL-encoded bodies (forms)
app.use(express.urlencoded({ extended: true }));

// Serve static files
app.use(express.static('public'));
```

Vestavěné middleware

Příklad s JSON body:

```
const express = require('express');
const app = express();

app.use(express.json());
app.post('/users', (req, res) => {
  console.log(req.body); // { name: 'John', age: 30 }
  res.json({ received: req.body });
});

app.listen(3000);
```

Vlastní middleware

Logging middleware - zaznamenává všechny požadavky:

```
function logger(req, res, next) {
  const timestamp = new Date().toISOString();
  console.log(`[${timestamp}] ${req.method} ${req.path}`);
  next();
}

app.use(logger);
```

Vlastní middleware

Autentizační middleware - kontroluje přihlášení:

```
function authenticate(req, res, next) {
  const token = req.headers.authorization;

  if (!token) {
    return res.status(401).json({ error: 'No token' });
  }

  // Verify token...
  next();
}
```

Vlastní middleware

Použití middleware pro konkrétní routes:

```
// Apply to all routes
app.use(logger);

// Apply to specific route
app.get('/admin', authenticate, (req, res) => {
  res.send('Admin panel');
});

// Apply to route group
app.use('/api', authenticate);
```

Error handling middleware

Error handling middleware má **4 parametry**:

```
function errorHandler(err, req, res, next) {
  console.error(err.stack);
  res.status(500).json({ error: 'Something went wrong!' });
}

// Must be last middleware
app.use(errorHandler);
```

Error handling middleware

Error handling middleware má **4 parametry**:

```
function errorHandler(err, req, res, next) {
  console.error(err.stack);
  res.status(500).json({ error: 'Something went wrong!' });
}

// Must be last middleware
app.use(errorHandler);
```

Důležité: Error handler musí být **poslední** middleware!

Kvízová otázka

Kvízová otázka

Jak Express ví, že to, co mu předáváme do `app.use(...)`, je middleware nebo error handler?

Kvízová otázka

Podle **arity** (počtu parametrů)

Kvízová otázka

Podle **arity** (počtu parametrů)

```
function middleware(req, res, next) { }

function errorHandler(err, req, res, next) { }
```

Kvízová otázka

Podle **arity** (počtu parametrů)

```
function middleware(req, res, next) { }

function errorHandler(err, req, res, next) { }

console.log(middleware.length);   // 3
console.log(errorHandler.length); // 4
```

Error Handling

Jak předat chybu do error handleru:

```
app.get('/users/:id', (req, res, next) => {
  const user = findUser(req.params.id);
  if (!user) {
    const error = new Error('User not found');
    error.status = 404;
    return next(error); // Pass error to error handler
  }
  res.json(user);
});
```

Error Handling

Vylepšený error handler:

```
function errorHandler(err, req, res, next) {
  const status = err.status || 500;
  const message = err.message || 'Internal Server Error';

  console.error(`[ERROR] ${status}: ${message}`);

  res.status(status).json({
    error: message, status: status
  });
}
```

Shrnutí

Klíčové poznatky

Co jsme se dnes naučili:

Klíčové poznatky

Co jsme se dnes naučili:

- **Express.js** je minimalistický framework nad Node.js

Klíčové poznatky

Co jsme se dnes naučili:

- **Express.js** je minimalistický framework nad Node.js
- **Routing** – `app.get/post/put/delete` a route parametry

Klíčové poznatky

Co jsme se dnes naučili:

- **Express.js** je minimalistický framework nad Node.js
- **Routing** – `app.get/post/put/delete` a route parametry
- **Middleware** – funkce `(req, res, next)`, vykonávají se v pořadí registrace

Klíčové poznatky

Co jsme se dnes naučili:

- **Express.js** je minimalistický framework nad Node.js
- **Routing** – `app.get/post/put/delete` a route parametry
- **Middleware** – funkce `(req, res, next)`, vykonávají se v pořadí registrace
- **Error handler** poznáme podle arity – má 4 parametry

Klíčové poznatky

Co jsme se dnes naučili:

- **Express.js** je minimalistický framework nad Node.js
- **Routing** – `app.get/post/put/delete` a route parametry
- **Middleware** – funkce `(req, res, next)`, vykonávají se v pořadí registrace
- **Error handler** poznáme podle arity – má 4 parametry
- Chybu předáme dál pomocí `next(error)`

Klíčové poznatky

Zatím jsme psali čistý JavaScript

Klíčové poznatky

Zatím jsme psali čistý JavaScript

Příště přidáme **TypeScript** – typované routy, validaci dat a strukturu většího projektu